

sotOs: Low-Detectability High-Interaction Deception via Real-Software Fidelity on a Capability Microkernel

Abstract— In high-interaction honeypots, detectability and fragile containment are coupled limitations: raising a decoy's fidelity introduces observable divergences and weakens isolation, whereas hardening isolation produces the seams by which honeypots are identified, such as unsupported system calls and TLS/TCP fingerprints inconsistent with the advertised system. This paper proposes an architecture that decouples the two objectives. Rather than adding containment on top of a high-fidelity operating system, sotOs builds fidelity on seL4, a microkernel whose isolation rests on machine-checked proofs (our prototype adds two narrow, auditable syscall-path patches that leave the capability model unmodified; §VI), by running real, unmodified software. Observable indistinguishability and structurally enforced containment can coexist, as we show experimentally. Each decoy, a sotBox, runs as a capability-confined task that can touch only what is explicitly delegated to it, while a user-space Linux ABI layer, lucAs, executes stock, dynamically linked musl/Alpine binaries with no special cases: busybox, CPython, SDL2, GTK3, and Doom, plus an in-progress Wine bring-up that loads and executes ntdll, a path toward unmodified Windows binaries (§IV). Two results anchor the evaluation. First, a differential fidelity discipline runs the reconnaissance battery issued in the first minute against a reference Alpine and closes the structural tells it exposes, from byte-exact JA3S/JA4S TLS and TCP/IP fingerprints to persona coherence, libc-internal interfaces, and eliminating spurious ENOSYS returns. Second, hosting Wine on a non-Linux capability microkernel exercises the Linux ABI harder than any other guest we run. The governing thesis is that compatibility is the multiplier of deception: what makes the host credible to a human also convinces an automated scanner, and the capability substrate contains a compromised decoy by construction rather than by chance.

Keywords— cyber deception, anti-fingerprinting, high-interaction honeypot, honeypot evasion, seL4, Linux ABI, Wine

I. PROBLEM STATEMENT AND APPROACH

A high-interaction honeypot must deliver the attacker's behavior to the defender, namely the commands executed, the tools deployed, and the lateral movement attempted, all observed within a controlled environment. This is achievable only if the environment is convincing enough that the attacker treats it as a real target, and contained enough that running real code grants no pivot. The two goals pull in opposite directions: plausibility demands a real operating system running real software, whereas safe containment demands a minimal, verifiable substrate with strong isolation. Common designs concede one side or the other. A monitored Linux VM is a real operating system, but its containment boundary is the entire attack surface of the Linux kernel. A syscall-emulating sandbox contains well, but its emulation is precisely where detection lives, because real software exercises the corners that emulators leave as stubs.

Anonymized for review: source code, gate scripts, and reference captures are available at <https://github.com/7841qz/sotOs-anonymous>; the de-anonymized repository

This work begins from a testable hypothesis: if fidelity is obtained by running real software on a kernel whose isolation is formally verified, the seams that betray a honeypot disappear without sacrificing containment. On that basis, sotOs treats the Linux ABI as a surface to host with fidelity rather than to emulate defensively. The governing thesis is the following.

A. Compatibility is the multiplier of deception

A honeypot that runs the attacker's unmodified tools, their busybox, their Python, their scanner, and eventually their Windows malware under Wine[2], is harder to detect than one that hand-codes responses, because it leaves fewer special cases to stumble over.

B. Threat model and objectives

We assume a remote attacker, human or automated, who reaches sotOs over SSH or HTTPS, fingerprints it, obtains a shell, performs reconnaissance, and attempts to deploy tooling, from native Linux binaries through, ultimately, Windows malware, all while actively probing for signs that the system is a honeypot or a sandbox. In the present prototype, Linux binaries execute with full fidelity (§IV); Windows execution is the target of an in-progress Wine bring-up (§III) and is not yet supported end to end. The host must meet four objectives. *Deceive*: present, across the observable layers it probes, a system indistinguishable under battery B (§VIII) from a real and somewhat neglected production box. *Contain*: run real code while ensuring that it cannot escape the sotBox, corrupt or exhaust the host, or reach real third parties. *Endure*: survive sustained attack and long-running workloads with no leaks, no privilege drift, and no anomalous faults. *Observe*: record everything without observable side effects. sotOs is not a general-purpose desktop operating system and does not pursue performance parity with Linux; the compatibility goal is fidelity to the ABI on which real software depends, and no more than that.

C. Architecture in brief

sotOs boots as an seL4 root task that delegates authority to an orchestrator (orch), which creates each decoy as a sotBox with its own address space and a private memory arena. Each guest's Linux system calls are delivered to lucAs, the Linux ABI layer, as faults and serviced out of process. Network deception is a separate user-space component (synth-srv), and graphics are handled by a Wayland compositor built from scratch. §V gives the full detail.

We frame this work as an architecture rather than a tool. Our contribution is not a particular honeypot binary but a design in which observable fidelity and provable containment are decoupled and then reconciled on a capability substrate; the components described here (orch, lucAs, synth-srv, the compositor, and sotTrace) are one realization of that design, and we report them as evidence that the architecture is buildable, not as the deliverable itself.

D. Contributions

The two results that should matter to an operator, with the remainder of the work supporting them, are:

- A differential fidelity discipline for anti-fingerprinting (§II): run the reconnaissance battery against a real Alpine system, compare against it, and close the structural tells it exposes, from byte-exact TLS and TCP/IP fingerprints to the libc-internal kernel interfaces.
- An initial Wine bring-up on seL4 (§III): the Wine loader and preloader execute, Windows-style address-space reservations succeed, and ntdll.so loads and runs; full Win32, wineserver, and TEB/PEB support remains future work. Even at this stage it exercises the Linux ABI more aggressively than any other guest we run, and running Wine on a non-Linux capability microkernel is, to our knowledge, novel.

Underlying both is capability-structured containment on a capability model whose isolation is formally verified [1], [5]. Our prototype applies two narrow syscall-path patches to seL4; these do not modify the capability model, IPC, or untyped memory management, but they touch source within the scope of the published proofs, so the binary we run is not covered by those proofs as stated. The containment argument therefore rests on the unmodified capability model together with manual inspection of two auditable patches (§VI).

II. SURVIVING RECONNAISSANCE: A DIFFERENTIAL-FIDELITY METHODOLOGY

Automated adversaries fingerprint a target before trusting it, and the first thing a careful human does in a fresh shell is check whether it is real. *sotOs* treats each such check as a test to pass, measured against a real reference system rather than guessed at.

A. Byte-Exact Network Fingerprints

TLS (JA3S/JA4S). The *ServerHello*, comprising the chosen cipher, the list and order of extensions, the EC point formats, and the session-ticket and extended-master-secret extensions, is emitted so as to produce a JA3S/JA4S hash identical to that of a reference *nginx:alpine*. One finding is worth highlighting: the server's cipher-selection policy is itself a fingerprint. Honoring the client's cipher order (*nginx*'s default behavior) versus imposing the server's preference is observable, so the selection logic must match as well.

TCP/IP coherence. The SYN-ACK and RST option blocks (MSS, window scale, SACK, timestamps), the DF bit, and the IP-ID progression match a reference Linux capture on a per-path basis. The IP-IDs of ICMP, UDP, and established TCP increment, whereas those of SYN-ACK and RST are zero with DF set, and the responses to probes (ECN echo, NULL/Xmas behavior) follow the reference operating system. All of this is measured from real captures rather than assumed.

B. A Hand-Built TLS 1.3 Server

Linking a TLS library would betray the host through the library's own fingerprint, so *sotOs* implements TLS 1.3 directly on primitives: ECDHE over X25519 and P-256/P-384, the full set of *nginx* cipher suites (AES-128-GCM, AES-256-GCM, ChaCha20-Poly1305), *HelloRetryRequest*, the RFC 8446 key schedule, and RSA-PSS certificate signatures. It interoperates with *openssl s_client -tls1_3* across all suites and with TLS 1.2 clients while preserving a byte-exact JA3S/JA4S fingerprint. Each layer was validated against an independent ground truth (RFC test vectors, NIST CAVP known-answer tests, and a Python cryptography oracle) so that a homegrown verifier would not silently share a bug with the implementation it checks.

C. A Real SSH Transport and Shell

The service plane exposes the ports of a typical production host, each served by a responder that speaks the real protocol. On port 22, a complete SSH-2.0 implementation provides curve25519-sha256 key exchange, AES-128-CTR with HMAC-SHA-256 under the RFC 4253 KDF, public-key host authentication (including the easily overlooked requirement that SSH signs SHA256(H), not H, under RSASSA-PKCS1-v1.5), *userauth*, and the RFC 4254 connection protocol. A real *ssh* client completes the handshake, is rejected a configurable number of times, and then enters an interactive *busybox* shell whose *stdin* and *stdout* are wired to the SSH channel. The attacker runs *cat /etc/passwd*, *id*, *ls*, and *pipes*, and receives genuine canary content in return, encrypted, while writes are contained ("Permission denied") and the session is logged end to end. Whether the contents of that shell, that is, whether its reconnaissance surface withstands a careful adversary, is the subject of §II.D.

D. The Shell That Survives Reconnaissance

This is where most high-interaction honeypots fail. We run the same reconnaissance battery an attacker types in the first minute, against a reference Alpine 3.20 and against the *sotOs* honey shell over SSH, and we treat every structural divergence (a tool reported "not found," a system call returning *ENOSYS*, a response wrong in type rather than in value) as a tell to close. Each of the following classes is a seam that a careful human, or an automated anti-honeypot check, would probe, and each fix preserves containment.

- Persona coherence, per tier. A single identity must hold across *uname*, */etc/os-release*, */etc/alpine-release*, */etc/hostname*, */proc/version*, and */sys/kernel/**, and it is tier-dependent: the Tier-2 canary presents a production-database persona (*prod-db-01*), whereas trusted compatibility guests present the host's real Alpine identity, so the *nodename* reported by *uname -n* must equal the */etc/hostname* read by the same session. Our differential harness caught a regression in which making the compatibility persona coherent silently desynchronized the canary's, precisely because *uname -n* versus *hostname* is one of the checks a suspicious attacker runs.
- Missing tools, synthesized and contained. A real Alpine always has its package manager; the absence of *apk* is a categorical tell. *sotOs* serves *apk* (and *getconf*) as containment-respecting facades: byte-credible output for read-only probes (*apk --version*, *apk info*), and for mutating operations the real "no repository cache" error, which reads as a network-restricted box and contains the attacker, since no real installation occurs. The facade is a *#!* script that resolves to the existing shell, so it adds no code to the trusted base.
- libc-internal interfaces. *realpath(3)*, which sits behind *realpath*, *readlink -f*, and countless build tools, canonicalizes by opening the target with *O_PATH* and reading the magic link */proc/self/fd/N*; without that link it fails with *EINVAL*. Serving the per-descriptor link closes an entire family of failures at once. This is the recurring shape: a visible tool breaks because of an invisible kernel interface it depends on, found only by tracing the real binary.
- Contained symlinks. Symlinks the attacker creates (*ln -s* in writable space) are tracked per session so that *readlink* and *ls -l* behave as on Linux, but *open* does not follow them; a symlink to */etc/shadow* still

cannot be read through the link. Fidelity and containment are reconciled by exposing the edge's metadata without ever traversing it.

- ENOSYS elimination. A real kernel rarely returns ENOSYS for a common system call; having ls and id call llistxattr and receive ENOSYS is a tell visible under strace, closed with a credible no-op (an empty extended-attribute list) rather than left betraying the host under tracing.
- Version coherence. Even the busybox build banner must agree with the apk metadata (v1.36.1), down to the build-date string.

After these closures, the reconnaissance battery shows no structural divergence from a real Alpine across battery B (the first-minute reconnaissance and containment battery defined in §VIII): the shell is differentially matched, not merely plausible. The threat tier governs everything. A sotBox's privilege and what it is allowed to observe as true are both functions of its tier, so that "running real software with fidelity" (Tier-0) and "lying convincingly to malware" (higher tiers, up to a quarantine-and-collect state) are a single dial rather than two code paths. The same idea covers graphics and process reconnaissance: a credible /proc, a filesystem tree with planted canaries, and a "honey screenshot" of a plausible desktop, with /proc/self/exe returning the real path to trusted guests but the decoy to attacker shells.

Results — T1: TLS fingerprints. We exercised the sotOs TLS responder and a reference nginx:alpine (OpenSSL 3.5) with a fixed set of seven *openssl* clients (TLS 1.3 default, P-256 and P-384 groups, reordered suite list, AES-256, ChaCha20, and TLS 1.2 fallback), three runs per client, capturing the ServerHello with tcpdump and deriving JA3S and JA4S with a standard-library extractor. Across all seven clients, the JA3S and JA4S of sotOs matched those of nginx exactly on all three runs (Table 1): the sotOs TLS stack is indistinguishable at the ServerHello fingerprint level, under the tested seven-client matrix, from that of the system it claims to be, under both the classic passive fingerprint (JA3S) and its modern wire-ordered variant (JA4S). The active JARM fingerprint complements this evaluation and is reported as work in progress.

Client	Negotiated suite	JA3S	JA4S
TLS 1.3 · default, AES-256 (2 clients)	AES-256-GCM-SHA384	15af9...	t130200_1302_a56c5b993250
TLS 1.3 · P-256, P-384, reordered (3 clients)	AES-128-GCM-SHA256	f4feb...	t130200_1301_a56c5b993250
TLS 1.3 · ChaCha20 (1 client)	ChaCha20-Poly1305	475c9...	t130200_1303_a56c5b993250
TLS 1.2 · fallback (1 client)	EC D H E - R S A - A E S 2 5 6 - G C M - S H A 3 8 4	ec74a...	t120400_c030_12a20535f9be

Table1. Seven openssl client configurations, grouped into four ServerHello equivalence classes; all values identical to nginx:alpine and stable across three runs. JA3S shown as a 5-character prefix.

III. TOWARD WINE ON SEL4

The upper bound on compatibility addressed by this work is the execution of Windows software. The design decision that minimizes porting risk is to use Wine's native winewayland driver against the existing compositor, rather than to port an X server.

Building on the address-space reservation work (§VI), the bring-up cleared a chain of substrate gaps, each one a reusable fix: routing the execve of a real binary to the binstore, rather than to the shell's multi-call binary, and

propagating the environment through it; returning the executable's real path via /proc/self/exe and via readlinkat (to which must reduce readlink); and serving a real /usr/bin and a minimal /lib so that realpath resolves each path component. Finally, the dynamic loader had to be exposed as a file the process can open.

With these in place, the Wine loader runs, the wine-preloader runs and its Windows range reservations succeed through the frameless reservation path, and ntdll.so loads and executes. The current boundary is the segment-ABI and loader-bootstrap layer (the CPU state of the re-executed loader), followed by IPC with wineserver and the setup of the TEB and PEB through %fs/%gs. These remain in progress.

The methodological note generalizes: each wall was a missing ABI behavior on which real software depends, and resolving it removed a seam for every future guest, not only for Wine.

IV. THE COMPATIBILITY HOST: RUNNING REAL SOFTWARE

The compatibility thesis is validated by running real programs, each one stock, dynamically linked, with no application-specific code in sotOs; orch supplies only an environment. These programs serve a dual purpose as evaluation: they are the attacker's plausible tools, run to completion, not micro-benchmarks.

- busybox: an interactive sh -i honey shell with pipes, redirection, reconnaissance applets, and contained writes.
- CPython 3.12: the real interpreter (a heavy arena, the stdlib mounted from a zip) running attacker-supplied scripts, contained.
- SDL2 (software renderer): Alpine's unmodified SDL2 driving its Wayland backend over wl_shm, without EGL.
- Doom: doomgeneric and Chocolate Doom rendering real frames, first to a framebuffer and then onto the Wayland compositor through dynamic SDL2.
- GTK3: Alpine's unmodified gtk3-demo rendering a decorated demo-browser window over wl_shm, exercising client-side decorations, theming, fonts, image loading, and pointer-driven growth of the cursor pool. Hundreds of real frames are rendered, with committed pixels matching the client's cairo output and no faults.

Each of these surfaced reusable substrate: a real VFS with symlinks, unique inodes so that musl's loader does not deduplicate distinct libraries, byte-stream Wayland transport with SCM_RIGHTS, xdg_shell, read-only MAP_SHARED file mmap, a statfs ABI fix, restoration of the futex wake registration, and a heavy memory arena. Each of these benefits every guest, not only the program that motivated it. This is the thesis in miniature: the work of running one real program correctly removes a seam for all of them.

V. ARCHITECTURE

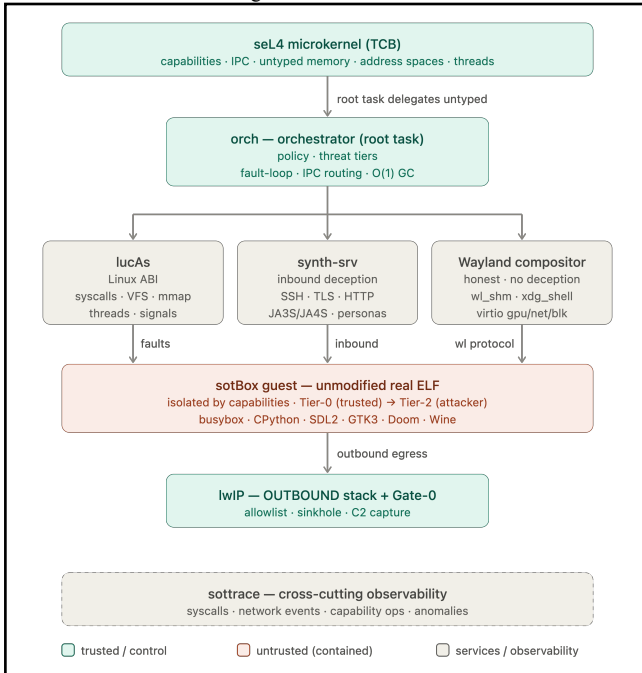
sotOs runs as a set of sel4 user components. For containment, the authority-owning TCB is hardware, sel4, and the root/orchestrator code that creates sotBoxes and delegates capabilities to them. lucAs, synth-srv, the compositor, and sotTrace are user-level components trusted for fidelity and availability rather than for the capability boundary: a compromised one of these can degrade deception or service, but cannot grant a sotBox authority beyond the capabilities orch explicitly delegated to it, since none of them holds authority it could pass on. lucAs

additionally enforces logical containment within those capabilities (for example, not following an attacker's symlink through open); a fault there could weaken that logical boundary without breaching the capability boundary. Figure 1 shows the component layout and the trust boundaries.

orch, the orchestrator. orch owns policy: it creates sotBoxes, assigns each a threat tier, supervises the fault loop, routes IPC between the guests and the deception and compositor[3] services, and reclaims cells. It is the single point that turns "a process faulted" into "a deception decision."

lucAs, the Linux ABI layer. Each guest is a real ELF in its own seL4 address space. Its Linux system calls are delivered to lucAs as faults and serviced out of process. lucAs provides real software with the surface it actually uses, along four fronts. The filesystem is a VFS with multiple backends: a baked-in, recursively populated /usr sysroot with real musl libraries and their symlinks, an in-memory writable tree, a procs, a PTY backend, POSIX shared memory, and a binstore.

Figure 1. Architecture



Dynamic linking goes through musl's ld.so inside the guest; sotOs places only the main image and the interpreter and backs the .so segments lazily, so that the closure of roughly 57 GTK3 libraries loads with no per-library special cases. Memory covers brk, anonymous and file-backed mmap, mprotect, guard pages, and frameless address-space reservations with MAP_FIXED commit-within-reservation for loaders such as Wine's preloader. Threads and signals are clone/pthread over shared address spaces with a real futex, together with a Linux signal surface.

synth-srv. synth-srv terminates inbound TCP and speaks the wire protocols byte for byte (§II). *The honest compositor.* This is a from-scratch Wayland compositor that speaks real wl_shm and xdg_shell over seL4 IPC, backed by virtio-gpu. It is honest in the sense that it serves real clients with full protocol fidelity; any deception lies in what is presented to hostile clients, not in a protocol infidelity. Guests' Wayland traffic is proxied by orch and lucAs, including reassembly of the sendmsg byte stream and the passing of SCM_RIGHTS descriptors for the shared-memory pools. *sotTrace.* sotTrace records system calls, network events, capability operations, and anomaly signals (§VII).

VI. CONTAINMENT: CAPABILITY-STRUCTURED AND RECLAIMABLE IN A SINGLE OPERATION

Running real, hostile code safely is a problem of memory and capability management, and this is where seL4 earns its place: containment does not depend on that code being free of bugs.

Per-sotBox arenas. Each sotBox's kernel objects are drawn from a single delegated untyped memory arena through a minimal, purpose-built allocator. Reclaiming a sotBox is a single seL4 CNode_Revoke of the arena's untyped, a single-operation reclamation (one revoke regardless of how much the sotBox allocated, O(1) in the operations the orchestrator issues, not in kernel time), sidestepping the LIFO free-path limits of a shared allocator. This is what lets sotOs rotate hundreds of spawn/teardown cycles where a naïve monotonic allocator jams after a dozen.

Zero leaks, measured. The last residue, a client-vspace accounting leak of a few frames per spawn, was closed with per-sotBox page-table-aligned windows whose arena-backed leaf frames are freed by the revoke. Under a soak harness the steady-state memory band is flat, survival is 100%, and the leak watermark is zero.

Anti-DoS. Per-cell fork/clone quotas freeze a fork bomb, nested ones included: the offending subtree is moved to a higher tier, its threads are suspended, and the subtree is reclaimed by the same arena revoke. CPU-spin and network-flood vectors are bounded analogously.

Address-space discipline. The low fixed reservations that real loaders need (Wine's preloader reserves the DOS area, a low window of roughly 1.7 GiB, and a high top-down window before the PE loader runs) are honored as frameless logical reservations, recorded but never committed as physical frames, with MAP_FIXED and mprotect committing only the touched subranges, and the non-fixed mmap allocator routing around the reserved windows.

Kernel-level patches. sotOs runs on an seL4 kernel with two narrow patches. The first preserves the user RSP across syscall entry (about 15 lines across traps.S, c_traps.c, and syscall.c), since the Linux syscall instruction does not save it and fault delivery would otherwise resume on a corrupted stack pointer. The second adds a per-TCB flag that routes all system calls from a Linux-ABI thread to seL4's existing UnknownSyscall fault mechanism, bypassing the fastpath. This is required by a register collision: Linux poll(timeout=-1) places -1 in the register seL4 reads as its syscall selector, where -1 is the valid SysCall selector, so the call would otherwise be dispatched as a capability invocation rather than delivered to lucAs. Neither patch modifies the capability model, IPC, untyped memory management, or the capability derivation tree, and the flag confers no new authority: a flagged thread holds exactly the capabilities it held before. Both patches do touch kernel source within the scope of the functional-correctness [1] and information-flow [5] proofs, so the binary we run is not covered by those proofs as stated; re-verifying these two patches is future work. The containment argument therefore rests on the unmodified seL4 capability model together with manual inspection of two narrow, auditable patches.

Why inspection suffices here. Both patches are confined to the syscall-entry and fault-dispatch path and touch none of the mechanisms the isolation proofs constrain: neither modifies the capability derivation tree, the access-control checks performed on capability invocation, the IPC transfer logic, or untyped retype/revoke. The first patch only preserves a caller register (RSP) across entry; it changes no control-flow decision and reads or writes no kernel object. The second changes a dispatch target, not an access decision:

when the per-TCB flag is set, a syscall that the fastpath would have interpreted as a capability invocation is instead delivered as an UnknownSyscall fault. Crucially, the flag adds no capability to the thread and removes none; a flagged thread can name and invoke exactly the capabilities it held before, and the UnknownSyscall path it is routed onto is itself an existing, in-proof seL4 mechanism, not new code. The information-flow argument is therefore unchanged in kind: any object a flagged thread can affect, it could already affect through the capabilities orch delegated to it, and the patch creates no new channel between sotBoxes, since dispatch routing is per-TCB and touches no shared kernel state. What manual inspection must establish is correspondingly narrow, that these two patches do not alter the cap-check, IPC, or memory-management code, which is a local and auditable property, rather than a re-proof of isolation.

The consequence for an operator is the following. Even when an attacker fully compromises a decoy, it holds capabilities only over the resources explicitly handed to that sotBox. There is no ambient authority to escalate toward, and the entirety of those resources is reclaimed by a single revoke.

VII. OBSERVABILITY

sotTrace records system-call entry and exit, network events, capability operations, tier transitions, and anomaly signals in per-source ring buffers under a global monotonic ordering, exportable as pcap (for standard tooling) and as event and flow graphs. A design goal is invisibility of the observation path, validated empirically for the channels we measure: the observation path introduces no guest-visible syscall or protocol artifact in our tests. Timing-channel invisibility is a goal but not yet measured; quantitative timing-side-channel analysis is future work. The validation campaign's certificates report the host as deceiving and containing across battery B, enduring under scaled endurance proxies (a literal multi-day run is future work, §X), and invisible with respect to syscall and protocol artifacts, with timing-channel analysis pending (§X), under the threat model of §I.

VIII. EVALUATION: GATE-DRIVEN DEVELOPMENT

Scope of fidelity claims. All fidelity claims in this paper are evaluated against a single reference system, Alpine 3.20, used as the oracle, under battery B. Battery B comprises the seven-client TLS matrix of Table 1, compared at the ServerHello fingerprint level, and the first-minute command set, compared at the level of output structure. Timing channels and a wider persona matrix are out of scope (§X). We use "indistinguishable" throughout in this measured sense: matched against this reference, under these probes.

Each capability sits behind a gate: a script that boots the image headless (or drives it through the emulator's input injection), exercises the capability, and asserts pass or fail against the serial log. Gates are the unit of truth: a feature is ready when its gate is green and the regression gates remain green. Debugging is empirical, that is, reproduce, instrument, refute a hypothesis against the reference ground truth, fix, and confirm no regression, rather than reasoning in the abstract; several subtle bugs in this work were found only because a live probe refuted a plausible reading of the code.

An adversarial escape-deny gate launches a battery of escape attempts (.. traversal, symlink-to-sensitive-file, /proc/self/fd, and writes to sensitive and persistence locations) and asserts that each is contained and that no real secret reaches the attacker through the tested escape vectors. This sharpened a distinction on which the deception model depends: an intentional honeypot, a fake /etc/shadow hash served as

bait, readable by a real root and meant to waste the attacker's time on cracking it, is a deception asset, not a leak. A secret detector that flags it mistakes the bait for a fault; the gate whitelists known honeytokens while still failing on any non-bait secret. The honeypot is the uncommon case in which the most convincing thing to show the adversary is exactly what a naïve leak detector forbids.

The deception surface is evaluated in the same way as the fingerprints: the reconnaissance battery is run on a reference Alpine 3.20 and on the honey shell over SSH, the outputs are compared per command, and the pass condition is zero structural divergence over battery B, with the escape-deny gate asserting containment over the same shell. Treating a real Alpine as the oracle makes reconnaissance fidelity a measured property rather than a matter of judgment.

Battery B. The evaluation battery comprises the probes an attacker issues in the first minute, in two parts with distinct pass conditions. The fidelity probes (pass condition: zero structural divergence from a reference Alpine 3.20) are: TLS ServerHello fingerprints across the seven OpenSSL client configurations of Table 1; TCP SYN-ACK/RST option probes (MSS, window scale, SACK, timestamps, DF, IP-ID); SSH login and interactive shell; persona checks across uname, hostname, /etc/os-release, /etc/alpine-release, /proc/version, and /sys/kernel/*; and shell/ABI checks including id, ls, cat, pipes, realpath, readlink -f, /proc/self/fd/N, apk --version, apk info, and xattr probes. The containment probes (pass condition: each contained, with no non-bait secret reaching the attacker) are: path traversal, symlink-to-sensitive-file, /proc/self/fd, and writes to sensitive and persistence locations.

IX. RELATED WORK (BRIEF)

Low-interaction honeypots are simple to deploy but trivial to fingerprint; high-interaction ones use real operating systems and pay for that realism in containment risk. Fingerprint detectability is documented: the systematic fingerprinting of low- and medium-interaction honeypots [4] shows that a single packet suffices to distinguish them from the system they claim to impersonate, precisely because their stock libraries implement the protocols in subtly different ways. sotOs attacks that problem at its root by running the real software. sotOs occupies a new point in the design space: real-software fidelity over an isolation model backed by machine-checked proofs [1], [5] (see §VI for the relationship to our patched kernel), with fingerprint fidelity as a first-class objective, supported by TLS fingerprinting techniques such as JA3/JA3S [6]. seL4's verified isolation makes it an attractive TCB; sotOs builds a Linux ABI personality on top, in the spirit of multiserver and personality-based systems but aimed at deception and contained execution. Syscall-emulation sandboxes [7] share lucAs's mechanism but invert the objective: where a sandbox narrows and stubs in order to contain, sotOs widens and implements with fidelity in order to resist the structural fingerprinting it is evaluated against, pushing containment down into the capability layer beneath. The broader discipline of deception is framed by frameworks such as MITRE Engage [8]. Running Wine on a non-Linux capability host is, to the best of our knowledge, novel.

The lineage of these ideas predates the present framing. Early accounts of engaging an intruder in an instrumented environment [9] established deception as an operational practice, and the honeypot was later formalized as a security construct [10]. Subsequent frameworks made deployment systematic, from virtual honeypot infrastructures that emulate many hosts on one machine [11] to widely used medium-interaction services such as the Cowrie SSH/Telnet honeypot [12]. These systems prioritize breadth of coverage

and ease of deployment, and it is precisely their reliance on emulated or stock service implementations that the fingerprinting results above [4] exploit. sotOs takes the opposite trade: a single host presented with full real-software fidelity rather than many emulated ones, accepting narrower coverage in exchange for removing the structural tells on which detection depends.

X. LIMITATIONS AND FUTURE WORK

- *Validation scale.* The endurance results reported here are scaled proxies; a literal multi-day run in KVM-backed CI is future work. Concurrent multi-sample containment (running several live malware samples in parallel sotboxes) is planned but not yet evaluated.
- *Trusted computing base.* sotOs applies two narrow patches to the seL4 kernel (§VI). They do not alter the capability or isolation model, but they touch source within the scope of seL4's published proofs [1], [5]; re-verifying them is future work.
- *Wine is mid bring-up (§III);* a complete Win32 GUI under winewayland is the next major milestone.
- *Graphics acceleration* is software-only by design (no EGL/GL); a DRM/GBM path is deferred.
- *Fingerprint coverage* is validated against specific reference captures; a wider matrix of personas would strengthen the claim.
- *Terminal fidelity* is validated against one reference persona (Alpine 3.20) over its probed command set. Some closures are deliberately shallow: the busybox banner is re-stamped rather than rebuilt at 1.36.1 (so applet-list parity is approximate), and contained symlinks expose metadata without being followed by open. A larger battery and a behavioral differential (not output-only) are future work.
- *The deception services' own TCB* (synth-srv and the protocol responders) runs in user space but warrants the same adversarial review as the containment core.

XI. CONCLUSION

sotOs argues, and demonstrates in working software, that the two classic enemies of high-interaction deception, namely detectability and the containment risk of running real code, are best addressed together rather than in isolation. By

hosting the adversary's real, unmodified tools on a capability-based microkernel and matching the impersonated system byte-for-byte at the network fingerprint level and structurally at the ABI layer, across the reconnaissance battery of §VIII, the host removes the seams exercised by our reconnaissance, ABI, and fingerprinting batteries while maintaining a hard containment boundary, reclaimable in a single orchestrator operation, around everything it runs. Compatibility is not a detour from deception; it is its sharpest instrument.

REFERENCES

1. G. Klein et al., "seL4: Formal verification of an OS kernel," in *Proc. 22nd ACM Symp. Operating Systems Principles (SOSP)*, Big Sky, MT, USA, 2009, pp. 207–220, doi: 10.1145/1629575.1629596.
2. The Wine Project. "WineHQ — Run Windows applications on Linux, BSD, Solaris and macOS." Accessed: Jun. 27, 2026. [Online]. Available: <https://www.winehq.org/>
3. Wayland Project. "The Wayland protocol." Accessed: Jun. 27, 2026. [Online]. Available: <https://wayland.freedesktop.org/>
4. A. Vetterl and R. Clayton, "Bitter harvest: Systematically fingerprinting low- and medium-interaction honeypots at Internet scale," in *Proc. 12th USENIX Workshop on Offensive Technologies (WOOT)*, Baltimore, MD, USA, 2018.
5. T. Murray et al., "seL4: From general purpose to a proof of information flow enforcement," in *Proc. IEEE Symp. Security and Privacy (S&P)*, Berkeley, CA, USA, 2013, pp. 415–429.
6. J. Althouse, J. Atkinson, and J. Atkins. "TLS fingerprinting with JA3 and JA3S." Salesforce Engineering. Accessed: Jun. 27, 2026. [Online]. Available: <https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967/>
7. E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "The true cost of containing: A gVisor case study," in *Proc. 11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, Renton, WA, USA, 2019.
8. MITRE Corporation. "MITRE Engage: An adversary engagement framework." Accessed: Jun. 27, 2026. [Online]. Available: <https://engage.mitre.org/>
9. C. Stoll, *The Cuckoo's Egg: Tracking a Spy Through the Maze of Computer Espionage*. New York, NY, USA: Doubleday, 1989.
10. L. Spitzner, *Honeypots: Tracking Hackers*. Boston, MA, USA: Addison-Wesley, 2002.
11. N. Provos, "A virtual honeypot framework," in *Proc. 13th USENIX Security Symp.*, San Diego, CA, USA, 2004, pp. 1–14.
12. M. Oosterhof. "Cowrie SSH/Telnet honeypot." GitHub. Accessed: Jun. 27, 2026. [Online]. Available: <https://github.com/cowrie/cowrie>